

Security Audit

Aegis 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Aegis team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Aegis is a decentralized finance (DeFi) protocol built on Ethereum that offers a Bitcoin-backed stablecoin called YUSD. It utilizes a delta-neutral strategy—holding BTC collateral while hedging with perpetual futures—to maintain price stability and generate yield from funding fees. The platform emphasizes full transparency, with real-time proof-of-reserves and an intuitive dashboard, and is governed by a DAO with its own governance token (AEG), enabling community-driven protocol decisions

Project Name: Aegis

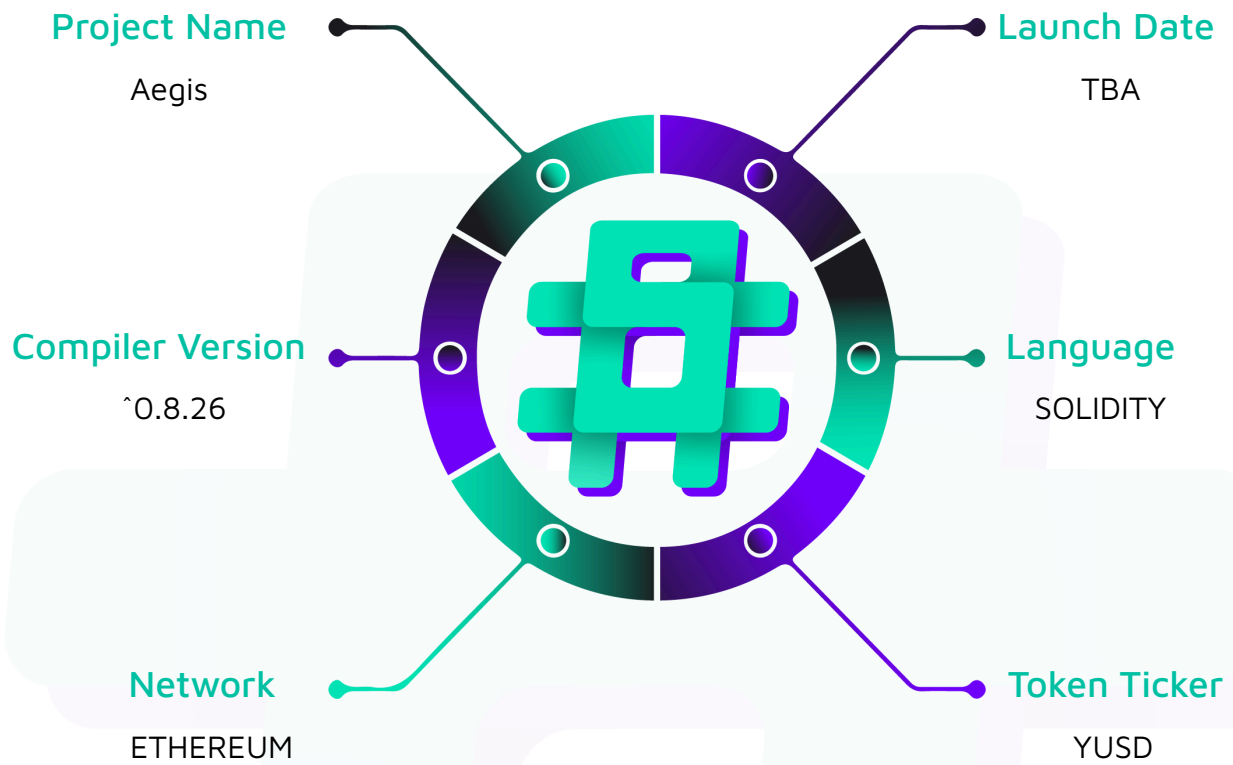
Project Type: DeFi

Compiler Version: ^0.8.26

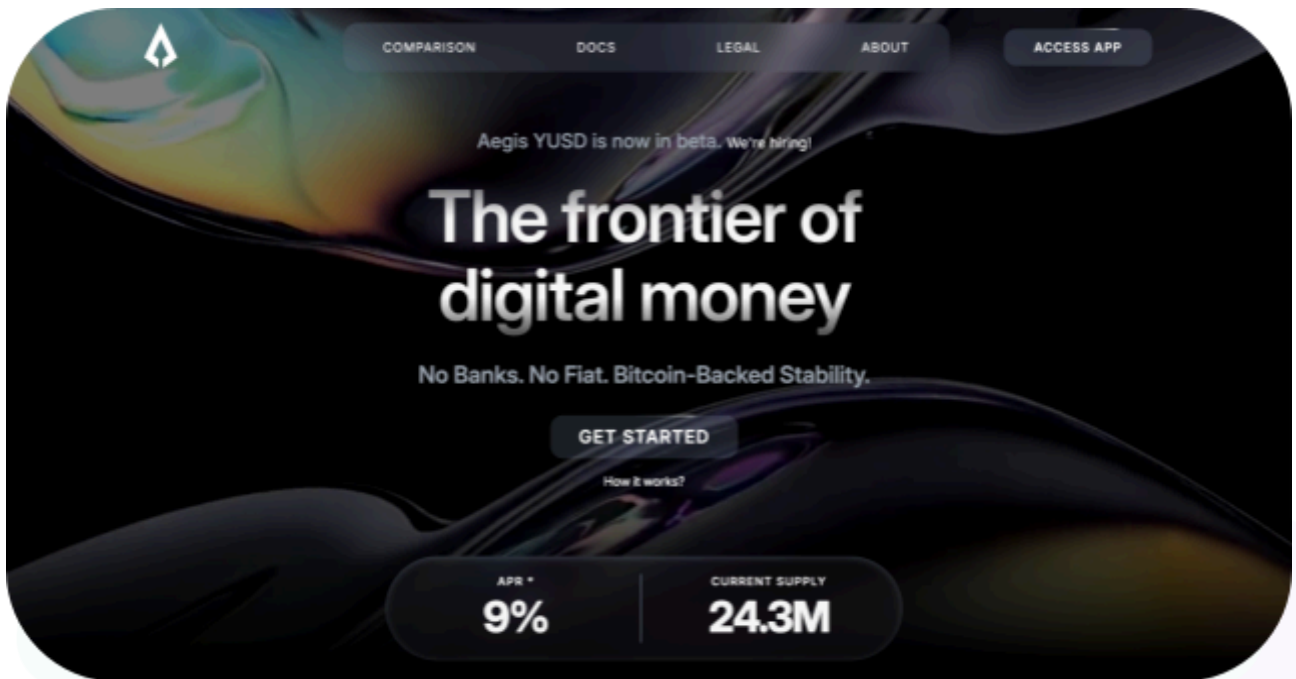
Website: <https://aegis.im/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Aegis project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Aegis Smart Contracts
Platform	Ethereum / Solidity
Audit Date	July, 2025
Contract 1	AegisMinting.sol
Contract 2	YUSDMintBurnOFTAdapter.sol
Contract 3	YUSDOFT.sol
Audited GitHub Commit Hash	31e6ff433203f6e2c928ec17aca80a525d00f257
Fix Review GitHub Commit Hash	3b6d9592a7df13bf303b5dab02d578a992dd65b2

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

3 Medium severity vulnerabilities

1 Low severity vulnerability

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AegisAdapter.sol - Allows users to: - Mint YUSD from Collateral Assets - Request Redeem of YUSD for Collateral - Withdraw Expired Redeem Request - Allows admins to: - Deposit income and mint rewards - Approve the redeem request and process the redemption - Reject Redeem Request - Transfer Assets to Custody Wallet - Force Transfer All Available Assets to Custody - Free and Unfreeze Asset Funds - Mint YUSD for Cross-Chain transfer - Burn YUSD for Cross-Chain transfer - Add/remove supported assets - Add/remove Custodian Address - Set AegisConfig, Cross-Chain Operator, and AegisOracle addresses.	Contract achieves this functionality.
YUSDMintBurnOFTAdapter.sol - Allows users to: - Initiate Cross-Chain transfer of YUSD(triggers debit/burn) - View token decimals, local decimals, and YUSD token address	Contract achieves this functionality.

YUSDOFT.sol - Allows users to: - Perform cross-chain token transfers using LayerZero - Execute standard ERC20 operations - View token metadata and LayerZero specific data	Contract achieves this functionality.
--	--

Code Quality

This audit scope involves the smart contracts of the Aegis project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Aegis project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] AegisMinting#mint - Incorrect Collateral Transfer Amount in Minting Process

Description

The mint function incorrectly transfers the full `order.collateralAmount` from the user instead of the calculated required collateral for the tokens being minted. This can lead to the over-collection of user funds or transaction failure.

Vulnerability Details

In the AegisMinting.sol contract, the `mint` function first determines the final amount of YUSD to be minted by taking the minimum of two different price sources. This final value, `yusdAmount`, represents the fair quantity of tokens the user should receive. However, on line 285, the contract executes a `safeTransferFrom` call using the original `order.collateralAmount` from the user's signed order. This amount can be significantly higher than the actual collateral required to mint `yusdAmount`, especially if market conditions have changed favorably for the user. Consequently, the contract takes more collateral than it should for the amount of YUSD it provides.

To illustrate the severity of this vulnerability, consider a user who wishes to mint 3,000 YUSD (assuming a target peg of \$1 per YUSD) by providing 1 WETH as collateral.

1. Order Creation: The user creates and signs an order specifying `collateralAmount = 1 WETH` and a desired `yusdAmount = 3000 YUSD`. At this moment, let's assume the Chainlink price for 1 WETH is exactly \$3,000. The order appears fair.
2. Favorable Price Movement: Before the transaction is mined, the market moves. The Chainlink price feed updates, and 1 WETH is now worth \$3,100. The user's collateral has become more valuable.

3. Correct Calculation (The 'Promise'): The `mint()` function executes and calls `_calculateMinYUSDAmount`. This function correctly identifies that at the new price, less than 1 WETH is needed to mint 3,000 YUSD. It correctly calculates the final `mintAmount` as 3,000 YUSD. Based on this, the user should only be charged approximately 0.967 WETH (\$3,000 worth) and keep the remaining collateral.
4. Flawed Execution (The 'Theft'): However, due to the bug on line 285, the contract ignores the result of its own fair calculation. Instead of transferring the required ~0.967 WETH, it transfers the full `order.collateralAmount`, which is 1 WETH.

Outcome: The user receives the 3,000 YUSD they requested but loses their entire 1 WETH. The protocol has effectively over-collected the collateral, pocketing the excess ~\$100 worth of WETH.

Impact

This critical flaw creates a direct financial risk for users, who may be overcharged and lose a portion of their collateral without receiving the corresponding value in minted tokens. This behavior effectively allows the protocol to "steal" excess funds. Furthermore, it creates a reliability issue, as minting transactions will unnecessarily revert if a user does not possess the full (and potentially outdated) `collateralAmount`, even if they have sufficient funds to cover the actual mint. This erodes user trust and compromises the core economic function of the protocol.

Recommendation

The contract's logic must be corrected immediately to ensure users are charged the correct amount of collateral. The `safeTransferFrom` call on line 285 should be modified to transfer the precise amount of collateral required for the final `mintAmount`. Following this, the state update to `_custodyTransferrableAssetFunds` on line 287 must also be updated to track this same corrected amount, ensuring the protocol's internal ledger remains accurate and consistent with the assets it actually holds.

Status

Resolved

Medium

[M-01] AegisMinting#onlyWhitelisted - Access Control Bypass in Whitelist Modifier

Description

The `onlyWhitelisted` modifier relies on the `aegisConfig.isWhitelisted` function, which contains a bug that allows bypassing whitelist validation when the whitelisted variable is set to false. This creates a critical access control vulnerability where unauthorized addresses might gain access to restricted functions.

Impact

A failure in the whitelisting mechanism could dismantle the protocol's entire access control system, allowing any user to execute sensitive functions like minting or redeeming YUSD. This could lead to uncontrolled token supply changes, manipulation of collateral pools, and a complete loss of the system's intended operational safeguards, severely compromising its security.

Recommendation

The underlying bug within the `aegisConfig.isWhitelisted()` function must be remediated to ensure it behaves correctly under all conditions, especially when whitelisting is disabled (it should fail-closed). As a defense-in-depth measure, robustly audit the configuration contract and consider adding redundant access control checks within `AegisMinting` itself.

Status

Acknowledged

[M-02] AegisMinting#mint - Incorrect Asset Accounting in Custody Fund Tracking

Description

The internal accounting for transferrable custody funds incorrectly increments by the user-supplied `order.collateralAmount` instead of the actual amount of collateral transferred, creating a ledger imbalance.

Impact

This accounting error leads to an inaccurate internal ledger of the protocol's available assets. This can cause downstream functions that rely on this accounting, such as collateral transfers to custody wallets or checks for redemption availability, to fail or operate on incorrect data, thereby undermining the financial integrity and reliability of the protocol.

Recommendation

To ensure accounting accuracy, correct the state update on line 287. The increment to `_custodyTransferrableAssetFunds` must use the same variable that reflects the actual amount of collateral transferred, guaranteeing consistency between the funds received by the contract and the funds it records.

Status

Resolved

[M-03] AegisMinting#_getAssetYUSDPriceOracle - Missing Negative Price Validation for Oracle

Description

The code retrieves `yusdUSDPrice` as an `int256` but doesn't validate that it's positive before potential conversion to `uint256`. Negative prices could be wrapped to extremely large positive values, causing calculation errors or unexpected behavior.

Impact

If the oracle reports a negative price, it would be cast to a massive positive number, causing price calculations for minting and redeeming to become wildly incorrect. This could be exploited by an attacker to drain collateral from the protocol by redeeming tokens for far more assets than they are worth, or it could cause legitimate user transactions to revert unexpectedly.

Recommendation

Add explicit validation for positive price value. Simply check `price > 0` instead of `== 0`.

Status

Resolved

Low

[L-01] AegisMinting#Constructor - Missing Zero Address Validation

Description

The `AegisMinting` constructor fails to validate that the `_aegisOracle` address is not the zero address, allowing the contract to be deployed in a non-functional state.

Deploying the contract with a null address for the oracle would cause all price-dependent operations, which are central to the contract's purpose, to fail. This would effectively render the contract's core functionality unusable and necessitate a costly and disruptive redeployment to correct the configuration error.

Recommendation

Implement preventative input validation in the constructor. Add a `require` statement to ensure that `_aegisOracle` and other critical contract addresses are not equal to `address(0)`. This simple check prevents fatal misconfigurations and ensures the contract is always initialized in a valid state.

Status

Acknowledged

QA

[Q-01] YUSDOFT - Floating Pragma Version

Description

The YUSDOFT.sol contract uses a floating pragma (^0.8.22), which risks introducing bugs or unintended behavior if the code is compiled with an unvetted future compiler version.

Recommendation

To ensure deterministic and secure builds, lock the pragma to a specific compiler version (e.g., pragma solidity 0.8.22;) in all production-level contracts. This guarantees that the compiled bytecode is always consistent and shielded from risks associated with unforeseen compiler changes.

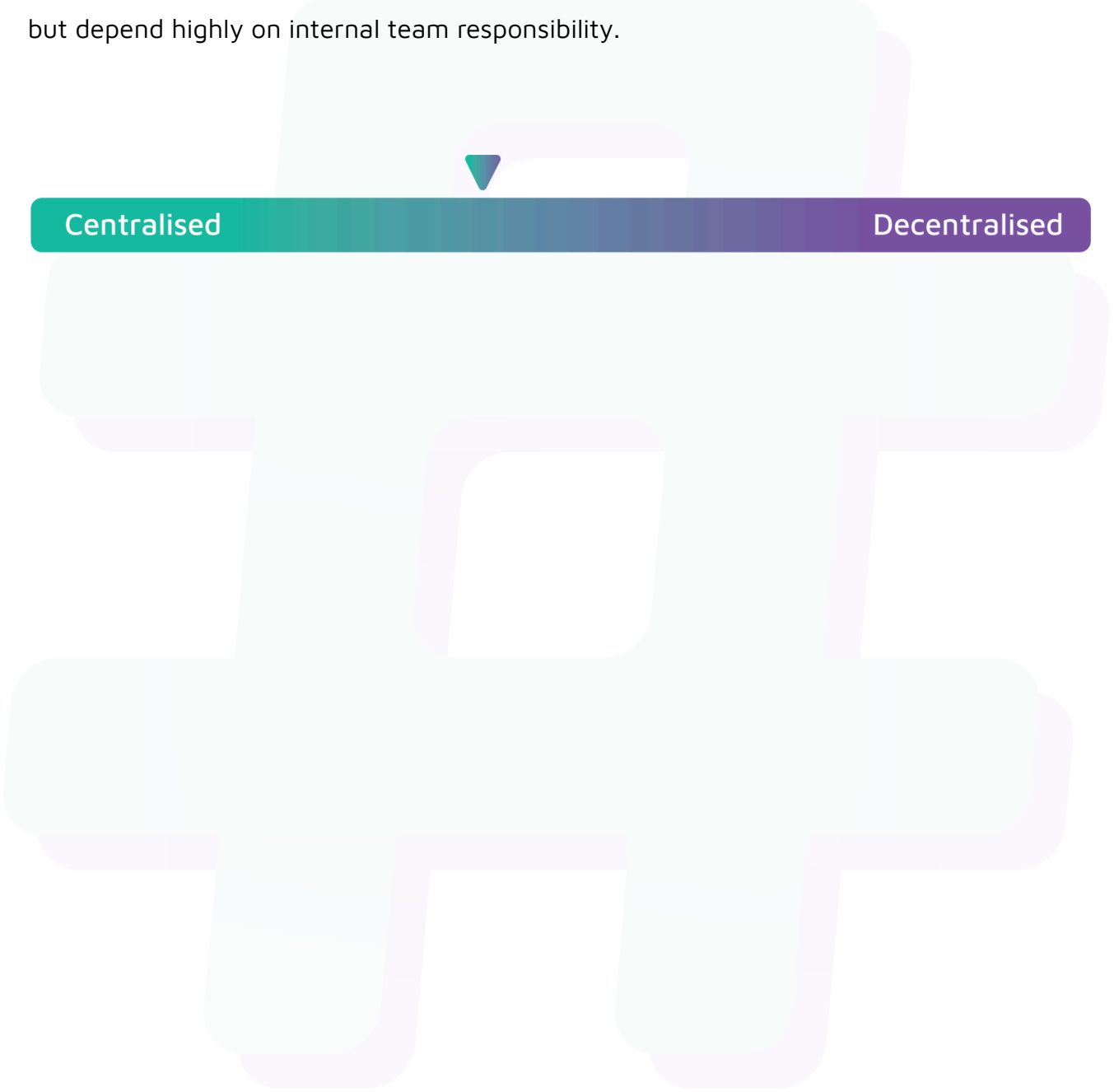
Status

Resolved

Centralisation

The Aegis project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Aegis project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.